

Docker Implementation for a Todo App

A practical beginner-friendly guide: Dockerfile → Image → Container → Docker Compose → React + Node.js + MongoDB

1. User Story

As a developer, I want to run my complete Todo application using Docker so that I don't need to manually install Node.js, MongoDB, or configure everything on every computer.

Without Docker:

```

graph TD
    A[Install Node.js] --> B[npm install]
    B --> C[Start React]
    C --> D[Install MongoDB]
    D --> E[Start MongoDB]
    E --> F[Configure backend]
    F --> G[Start Express]

```

With Docker:

```

docker compose up
    ↓
    █ Docker starts everything █
    █ React █
    █ Node/Express █
    █ MongoDB █
    █
  
```

2. Example Project

Assume the Todo application uses React for the frontend, Node.js + Express for the backend, and MongoDB for the database.

```

todo-app/
├── frontend/
│   ├── src/
│   ├── package.json
│   └── ...
├── backend/
│   ├── src/
│   ├── package.json
│   └── ...
└── docker-compose.yaml

```

3. Install Docker

Install Docker Desktop. Then verify the installation:

```
docker --version
docker compose version
```

4. Create a Dockerfile for the Backend

Inside the backend folder, create a file named **Dockerfile**:

FROM node:22

```
WORKDIR /app
```

```
COPY package*.json ./
```

```
RUN npm install
```

```
COPY . .
```

```
EXPOSE 5000
```

```
CMD ["npm", "start"]
```

Meaning: use Node.js 22 → create /app → copy package files → install dependencies → copy source code → document port 5000 → start the application.

5. Build the Backend Image

```
cd backend
```

```
docker build -t todo-backend .
```

Flow:

```
Dockerfile
```

```
↓ docker build
```

```
todo-backend IMAGE
```

6. Run the Backend Container

```
docker run -d -p 5000:5000 --name todo-backend todo-backend
```

Your Computer



localhost:5000



Docker



todo-backend container



Express server

7. Add MongoDB with Docker Compose

At the project root, create **docker-compose.yml**:

```
services:
```

```
  backend:
```

```
    build: ./backend
```

```
    ports:
```

```
      - "5000:5000"
```

```
    depends_on:
```

```
      - mongodb
```

```
  mongodb:
```

```
    image: mongo
```

```
    ports:
```

```
      - "27017:27017"
```

8. Connect Backend to MongoDB

When the backend runs inside Docker, do not use localhost to connect to MongoDB. Use the Compose service name.

```
mongodb://mongodb:27017/todo
```

The service is named **mongodb**, so Docker's internal network lets the backend reach it at **mongodb:27017**. Inside a container, localhost refers to that same container.

9. Add the React Frontend

Create **frontend/Dockerfile**:

```
FROM node:22

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 5173

CMD ["npm", "run", "dev", "--", "--host", "0.0.0.0"]
```

Then add the frontend service to docker-compose.yaml:

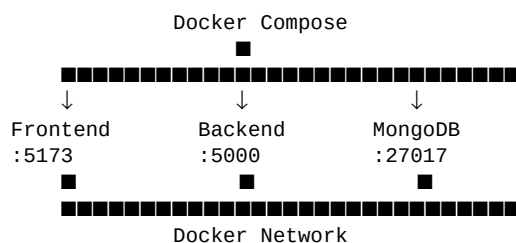
```
services:

  frontend:
    build: ./frontend
    ports:
      - "5173:5173"
    depends_on:
      - backend

  backend:
    build: ./backend
    ports:
      - "5000:5000"
    depends_on:
      - mongodb

  mongodb:
    image: mongo
    ports:
      - "27017:27017"
```

10. Complete Architecture



11. Start the Complete Application

```
cd todo-app
docker compose up -d
```

Check running services:

```
docker compose ps
```

View logs:

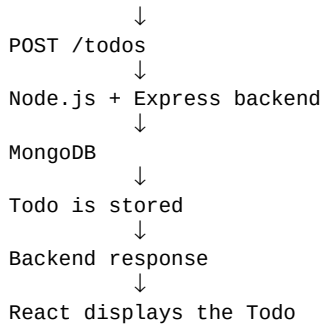
```
docker compose logs -f
```

Stop everything:

```
docker compose down
```

12. How a User Request Flows

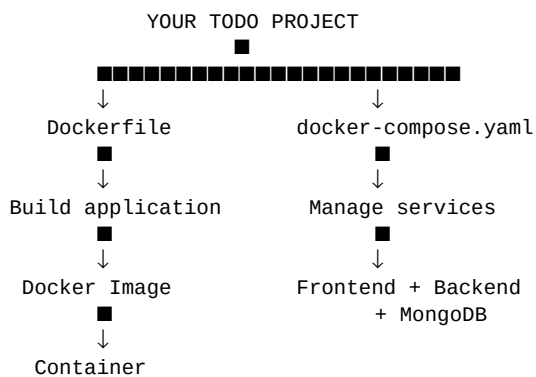
```
User opens http://localhost:5173
      ↓
React frontend
```



13. What Each File Does

File	Simple purpose
Dockerfile	Instructions for building your application's Docker image.
docker-compose.yaml	Defines and manages multiple services/containers and how they work together.
.dockerignore	Prevents unnecessary files such as node_modules and .git from being sent to the Docker build context.
.env	Stores configuration such as ports, database URLs, and secrets; do not commit real secrets to Git.

14. The Core Docker Mental Model



15. Daily Docker Workflow

Start:
`docker compose up -d`

Check:
`docker compose ps`

Logs:
`docker compose logs -f`

Stop:
`docker compose down`

16. Final Takeaway

Dockerfile builds your application's image. Docker Compose orchestrates your different services/containers. For the Todo app, the final setup is typically React + Node/Express + MongoDB, connected through a Docker network and started with one command: **`docker compose up -d`**.